Clase Abstracte si Interfete

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

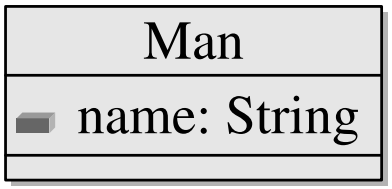
OOP, 2018



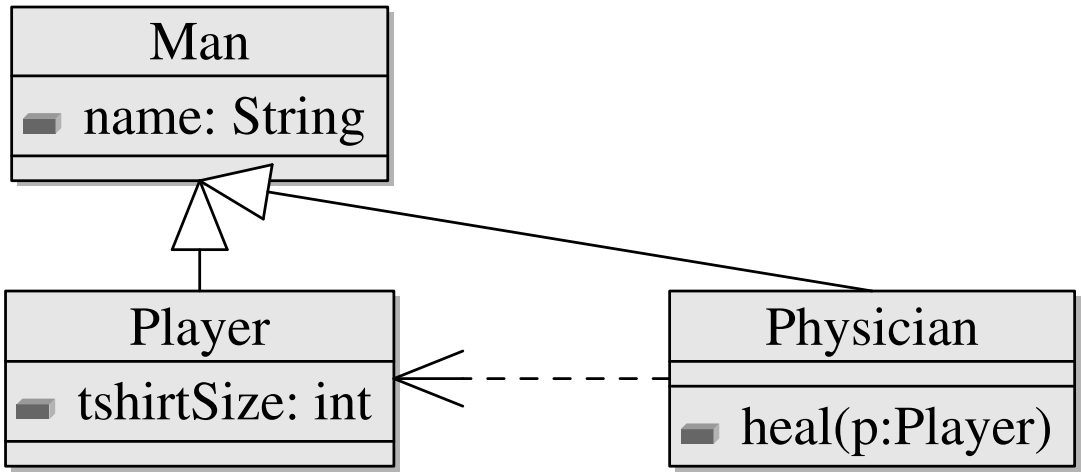




Curs 2: ce contine o clasa



Curs 3: relatia dintre clase

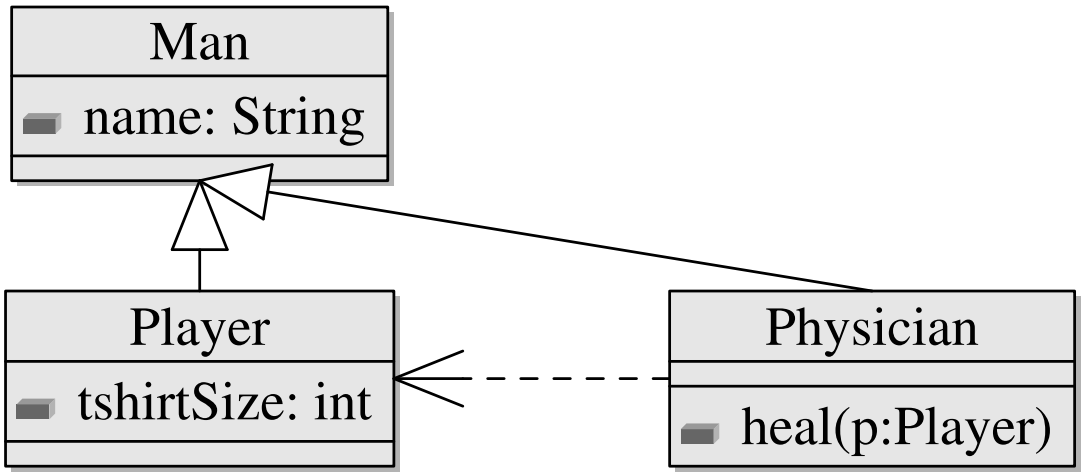


Exercitii retrospective

- google "uml class diagram": images
- ▸ Quizes

Interfete

Situatie in care e nevoie de interfete





Exemplu de interfata cu utilizatorul



Exemplu de interfata cu utilizatorul





Ce este o interfata?

Definition

Interfata este un tip de date care reprezinta, in forma sa de baza, o grupare de semnaturi de metode (fara implementari) unite de un scop comun.

Ce este o interfata?

Definition

Interfata este un tip de date care reprezinta, in forma sa de baza, o grupare de semnaturi de metode (fara implementari) unite de un scop comun.

```
public interface Payee {
  int calcMoney(Month paymentMonth);
}
public class Player implements Payee {
  private int noMatches;
  private Contract clubContract;
  public int calcMoney(Month paymentMonth) {
    return noMatches * clubContract.getPayPerMatch();
  }
}
public class Physician implements Payee {
  private Contract clubContract;
  public int calcMoney(Month paymentMonth) {
    return clubContract.getMonthlySalary();
  }
}
```

Ce poate contine o interfata?

- semnaturi de metode

Ce poate contine o interfata?

- semnaturi de metode
- metode statice (inclusiv implementari)
- constante (static final)

Ce poate contine o interfata?

- semnaturi de metode
- metode statice (inclusiv implementari)
- constante (static final)
- metode default (inclusiv implementari) NEW!! din Java 8

Ce poate contine o interfata?

- semnaturi de metode
- metode statice (inclusiv implementari)
- constante (static final)
- metode default (inclusiv implementari) NEW!! din Java 8

Toate sunt public in mod automat (nu se trec specificatori de acces)

Ce poate contine o interfata?

```
public interface Payee {

  int calcMoney(Month paymentMonth);

  static final String PAYER = "SuperStar Basket Club";
  static final Bank PAYING_BANK = "Rich Swiss Bank"
  static String printPayerData() {
    System.out.println(PAYING_BANK+" on the behalf of "+
        PAYER);
  }

  default Contract signContract() {
    return new Contract(PAYER);
  }

}
```

De ce sa folosim o interfata?

Interfata este un contract care defineste interactiunea a doua componente software. Se foloseste la decuplarea componentelor:

- cand doua echipe diferite lucreaza la cele doua componente

De ce sa folosim o interfata?

Interfata este un contract care defineste interactiunea a doua componente software. Se foloseste la decuplarea componentelor:

- cand doua echipe diferite lucreaza la cele doua componente
- cand vreau sa folosesc componenta si la altceva - ganditi-va la aplicatie HR pentru:

De ce sa folosim o interfata?

Interfata este un contract care defineste interactiunea a doua componente software. Se foloseste la decuplarea componentelor:

- cand doua echipe diferite lucreaza la cele doua componente
- cand vreau sa folosesc componenta si la altceva - ganditi-va la aplicatie HR pentru:
  - o alta echipa de baschet

De ce sa folosim o interfata?

Interfata este un contract care defineste interactiunea a doua componente software. Se foloseste la decuplarea componentelor:

- cand doua echipe diferite lucreaza la cele doua componente
- cand vreau sa folosesc componenta si la altceva - ganditi-va la aplicatie HR pentru:
  - o alta echipa de baschet
  - o echipa de fotbal

De ce sa folosim o interfata?

Interfata este un contract care defineste interactiunea a doua componente software. Se foloseste la decuplarea componentelor:

- cand doua echipe diferite lucreaza la cele doua componente
- cand vreau sa folosesc componenta si la altceva - ganditi-va la aplicatie HR pentru:
  - o alta echipa de baschet
  - o echipa de fotbal
  - o corporatie

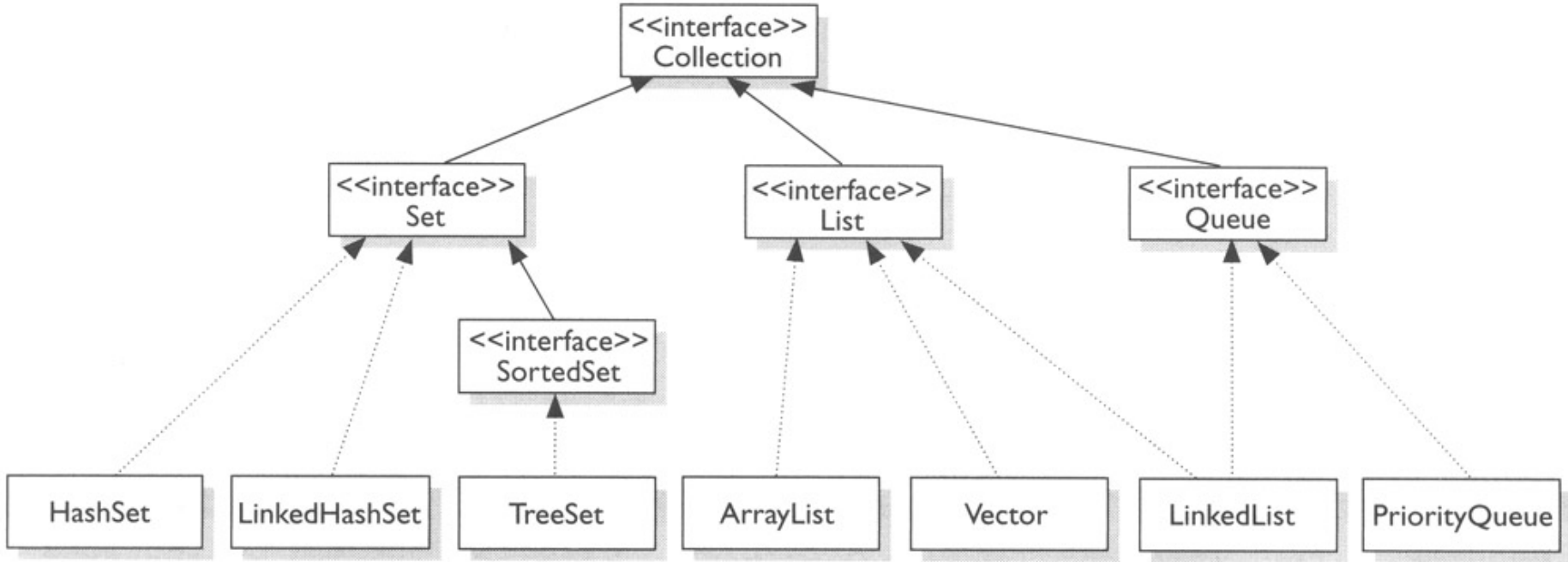
Interfete celebre: List

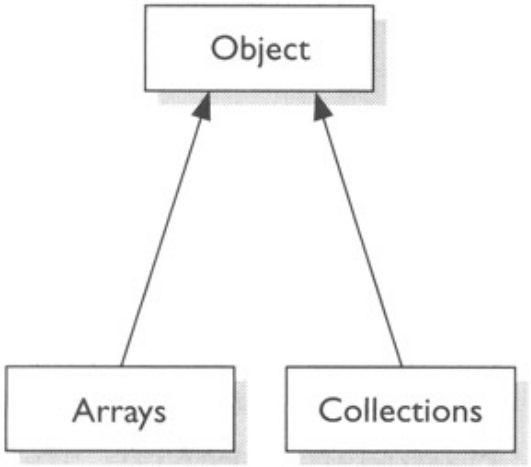
```
List<Player> allStarTeam = new ArrayList<Player>();
allStarTeam.add(new Player("Michael Jordan"));
allStarTeam.add(new Player("Kobe Bryant"));
allStarTeam.add(new Player("Shaquille O'Neal"));
allStarTeam.add(new Player("Magic Johnson"));
allStarTeam.add(new Player("LeBron James"));
```

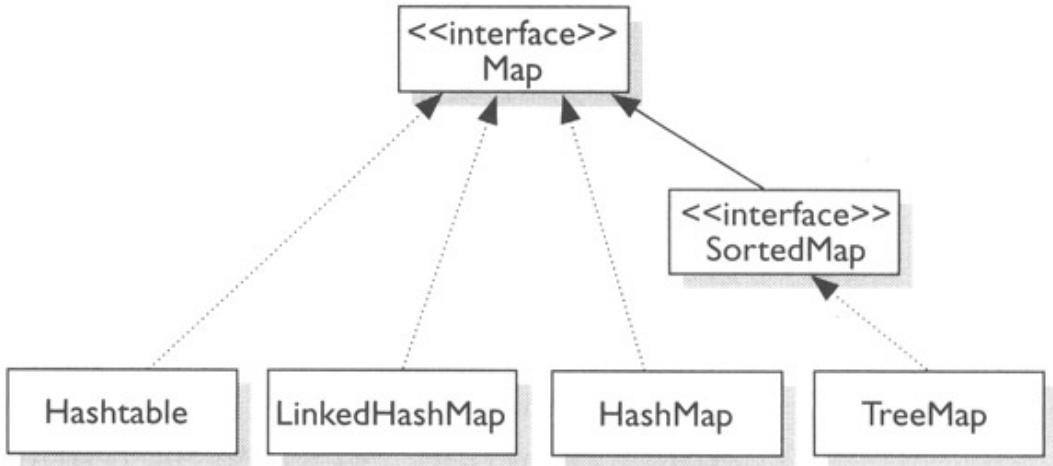
Interfete celebre: Comparable

```
public class Player implements Comparable {
    private float height;
    float getHeight() {
      return getHeight;
    }
    public int compareTo(Player otherPlayer) {
        return height - otherPlayer.getHeight();
    }
}
. . .
Collections.sort(allStarTeam);
```

Interfete celebre: o familie de colectii







Relatii intre interfete si intre interfete si clase

- o interfata poate extinde mai multe interfete (mostenire)
- o interfata poate fi implementata de mai multe clase
- o interfata nu poate fi instantiata
- o clasa poate implementa mai multe interfete

Evolving interfaces

```
public interface Payee {
  static final String PAYER = "SuperStar Basket Club";
  static final Bank PAYING_BANK = "Rich Swiss Bank"
  static String printPayerData() {
    System.out.println(PAYING_BANK+" on the behalf of "+
        PAYER);
  }
  default Contract signContract() {
    return new Contract(PAYER);
  }
}

public interface Employee extends Payee {
  int calcSalary(Month paymentMonth);
}

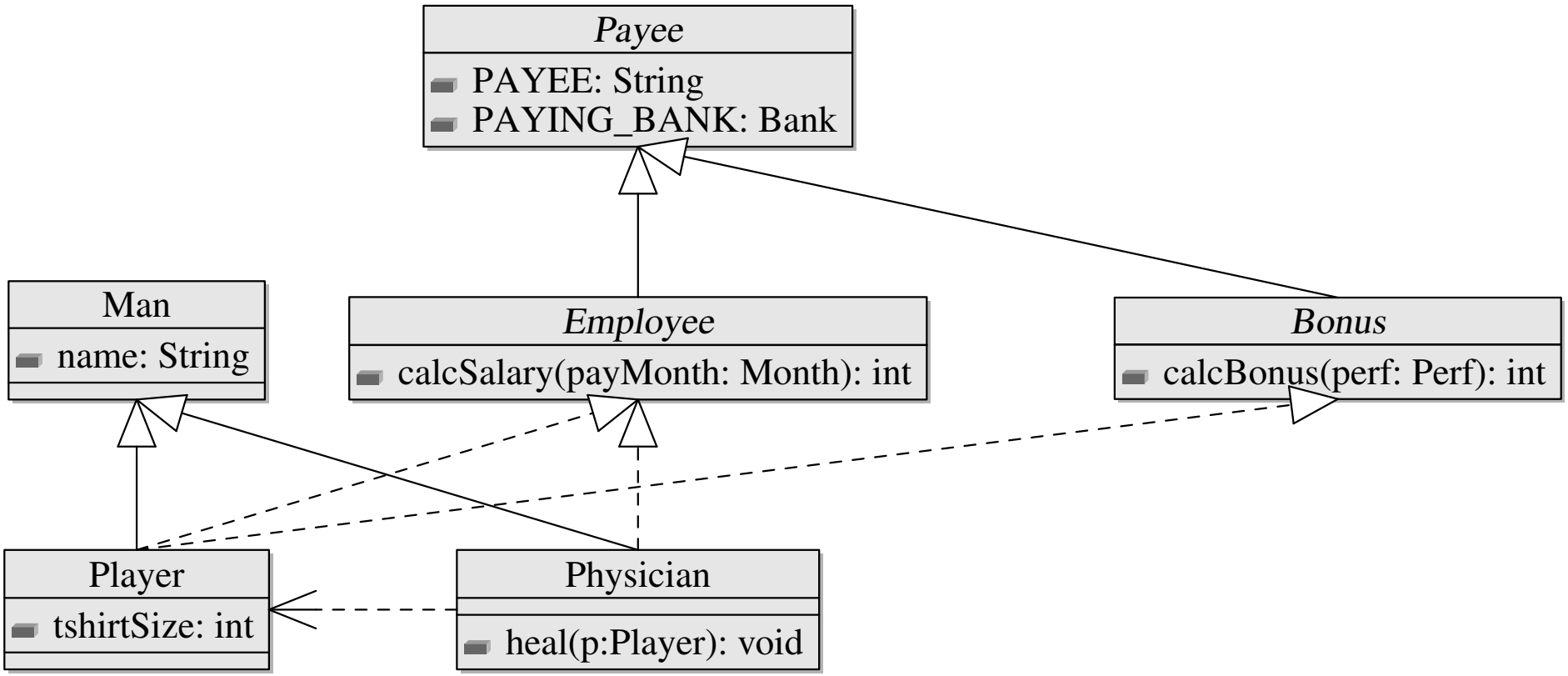
public interface Bonus extends Payee {
  int calcBonus(Performance performance);
}
```

Evolving interfaces

```
public class Player extends Man implements Employee, Bonus {
    ...
}

public class Physician extends Man implements Employee {
    ...
}
```

Evolving interfaces



Clase Abstracte

Nevoia unor entitati intermediare

Payee:

- signContract este valabil pentru toti cei care sunt platiti de companie, deci poate fi implementat din start

Nevoia unor entitati intermediare

Payee:

- signContract este valabil pentru toti cei care sunt platiti de companie, deci poate fi implementat din start
- calcMoney este variabil in functie de cei care extind Payee (Employee, Bonus) deci poate fi declarat abstract si lasat spre implementare acestora

Definition

Clasa abstracta este o clasa declarata ca abstracta prin utilizarea cuvantului cheie "abstract". Desi nu este obligatoriu, acest lucru are sens sa se intample daca cel putin una din metodele sale este abstracta (adica declarata fara implementare). Reciproca insa este obligatorie: daca o clasa are cel putin o metoda abstracta, toata clasa trebuie marcata ca abstracta.

Ce poate contine o clasa abstracta

- orice contine o clasa obisnuita, doar ca este marcata ca abstracta (de fapt o clasa obisnuita poate fi marcata ca abstracta fara vreun motiv anume)
- elegant ar fi ca cel putin una din metodele unei clase abstracte sa fie abstracta (adica semnatura sa aiba cuvantul cheie abstract si sa nu aiba implementare)

Specificatorii de acces se pastreaza ca la o clasa obisnuita.

De ce sa folosim o clasa abstracta?

Clasa abstracta ofera o cale de mijloc intre clase si interfete, oferind posibilitatea de a crea clase incomplete, care vin cu un contract pentru a fi complet implementate de clasele ce le mostenesc:

- cand definim intr-un mod corect ceea ce au in comun clasele derivate

De ce sa folosim o clasa abstracta?

Clasa abstracta ofera o cale de mijloc intre clase si interfete, oferind posibilitatea de a crea clase incomplete, care vin cu un contract pentru a fi complet implementate de clasele ce le mostenesc:

- cand definim intr-un mod corect ceea ce au in comun clasele derivate
- cand dorim sa oferim o implementare partiala unei clase, ca sa nu duplicam cod (desi din Java 8 acest lucru se poate face si cu interfete si metode default)

Famous example: AbstractList

- AbstractList reference

Relatii intre clase abstracte, interfete si clase

- o clasa poate extinde o clasa abstracta daca ii implementeaza toate metodele abstracte
- daca o clasa extinde o clasa abstracta si nu ii implementeaza toate metodele abstracte, atunci, la randul sau, va fi marcata ca abstracta

Mostenira multipla

Multiple inheritance

Definition

Mostenirea multipla este o functionalitate oferita de unele limbaje de programare prin care o clasa derivata preia caracteristicile mai multor clase parinte.

Multiple inheritance

Definition

Mostenirea multipla este o functionalitate oferita de unele limbaje de programare prin care o clasa derivata preia caracteristicile mai multor clase parinte.

In Java:

- o clasa poate extinde o singura alta clasa
- o clasa poate implementa mai multe interfete

Multiple inheritance

Definition

Mostenirea multipla este o functionalitate oferita de unele limbaje de programare prin care o clasa derivata preia caracteristicile mai multor clase parinte.

In Java:

- o clasa poate extinde o singura alta clasa
- o clasa poate implementa mai multe interfete

Din Java 8:

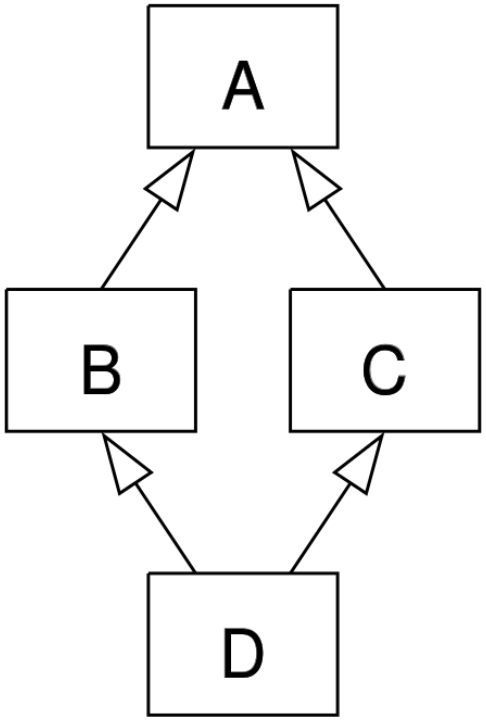
- o interfata poate contine metode default
- astfel, se poate implementa Mostenirea Multipla printr-o clasa care implementeaza mai multe interfete cu metode default

Problema diamant

Problema diamant ("diamond problem", aka "deadly diamond of death")

Problema diamant

Problema diamant ("diamond problem", aka "deadly diamond of death") se refera la ambiguitatea care apare atunci cand doua clase B si C mostenesc o clasa A, iar o alta clasa D mosteneste atat B, cat si C.



Daca exista o metoda in A, pe care o suprascriu atat B, cat si C, dar D nu o suprascrie, atunci D pe care din cele doua variante o va mosteni?

Problema diamant in Java 8 și inheritance prin interfaces

Programatorului îi este solicitat să rezolve ambiguitatea apelând B.super.m(...) sau C.super.m(...)

▸ What about the diamond problem? on lambdafaqs.org

Tipuri de mostenire multipla

- of State: not a problem, interface do not have fields
- of Implementation: default methods
- of Type: using an interface as a type

▸ Multiple Inheritance of State, Implementation, and Type

Mostenire multipla: interfete vs clase

Mostenire multipla: interfete vs clase

- se mostenesc implementari, nu si stare (campuri)
- nu se mostenesc constructori

▸ Multiple Inheritance of State, Implementation, and Type

Interfete vs Clase Abstracte

	Interfata	Clasa Abstracta
Se poate instantia	nu	nu
Metode	cu sau fara implementare	cu sau fara implementare
Membrii	static final	de orice fel
Specificatori acces	public (automat)	public, private, protected, default
Relatia cu alte clase	o clasa poate implementa oricate interfete	o clasa poate extinde o singura clasa abstracta

Cand sa folosim Interfete si cand Clase Abstracte?

Use abstract classes when:

- sharing code among several closely related classes, or
- classes that extend the abstract class have many common methods or fields, or require access modifiers other than public, or
- non-static or non-final fields are needed (methods that can access and modify the state of the object to which they belong)

Cand sa folosim Interfete si cand Clase Abstracte?

Use abstract classes when:

- sharing code among several closely related classes, or
- classes that extend the abstract class have many common methods or fields, or require access modifiers other than public, or
- non-static or non-final fields are needed (methods that can access and modify the state of the object to which they belong)

Use interfaces when:

- unrelated classes would implement the interface, or
- specifying the behavior of a particular data type, but not concerned about who implements its behavior, or
- multiple inheritance of type is needed

Pentru mai multe detalii ▸ Tutorialul despre Clase si Metode Abstracte

Ar trebui sa intelegeti ce scrie aici:

- Interviu 1
- Interviu 2

De citit

Este util sa cititi textul laboratorului inainte de laborator:

- Lab 4: Clase abstracte si interfete